

Learning & Remembering

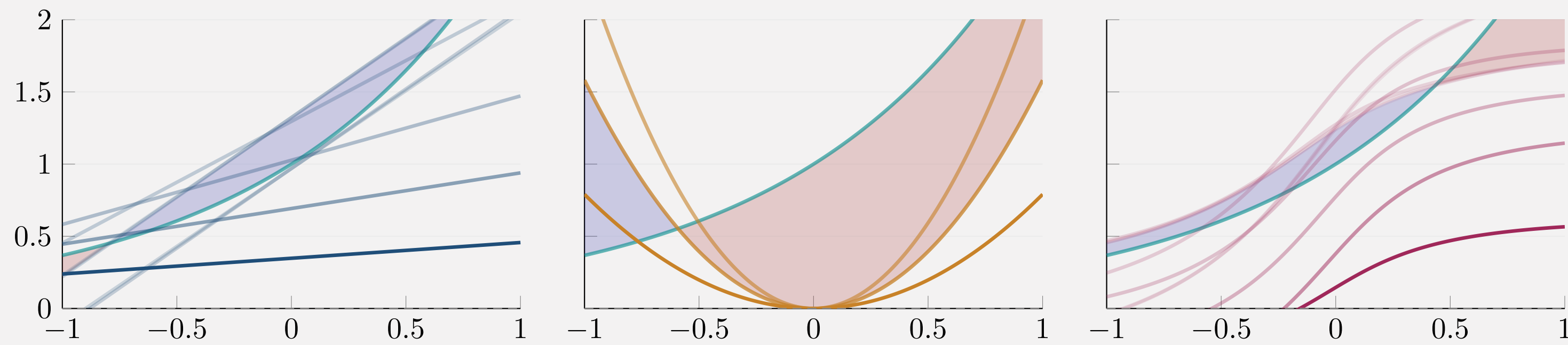
Problems are about actions. Learning should be too.

John C. Raisbeck · Bruno Castro da Silva

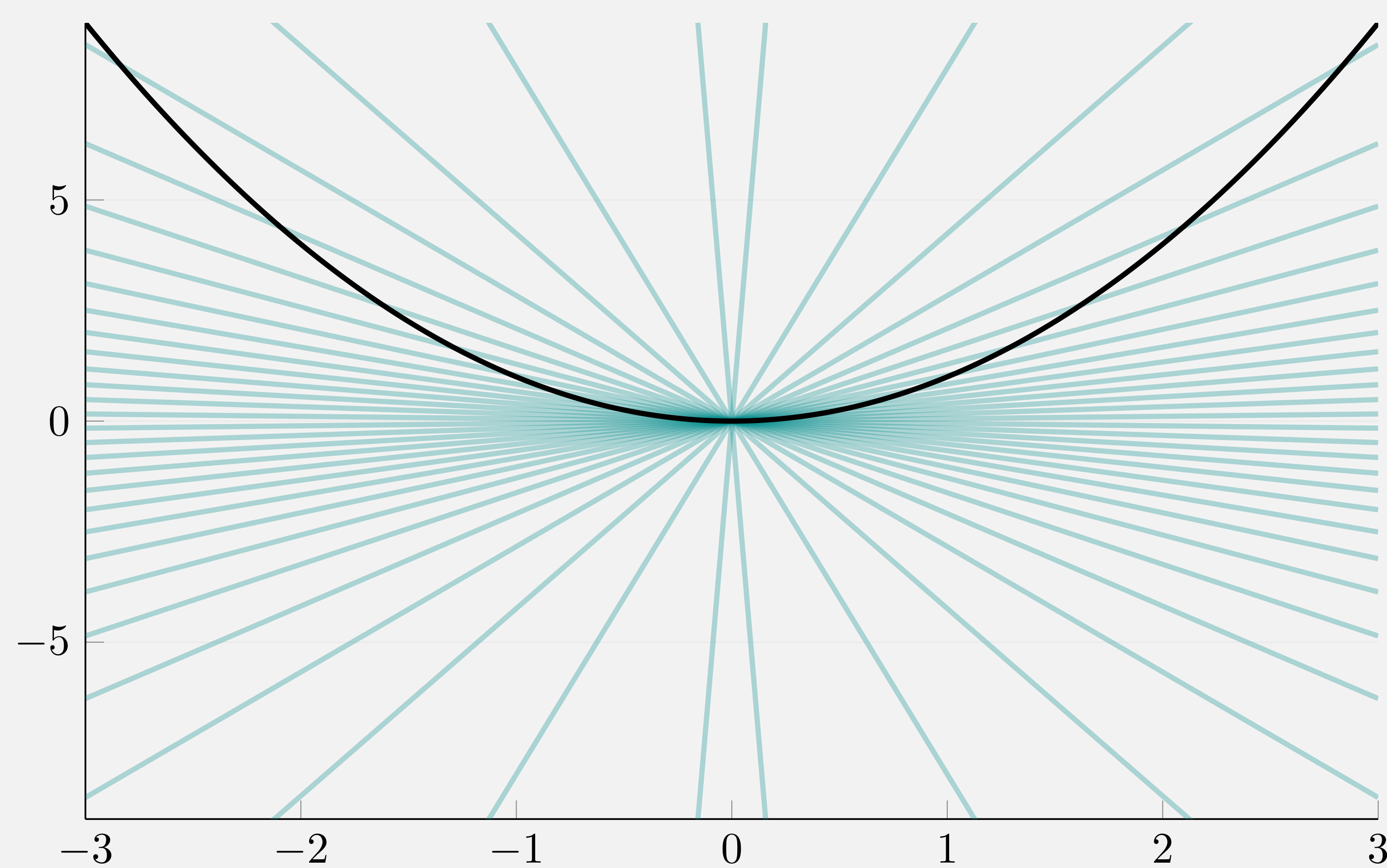
Manning College of Information & Computer Sciences, University of Massachusetts Amherst

What would you do?

Learning is the *change* in the function, $f_{t+1} - f_t$, after encountering a state. What the agent *should* learn is determined by the environment and the learning algorithm. What it *can* — and actually *does* — learn is restricted by the structure of its brain.



The functions f can express



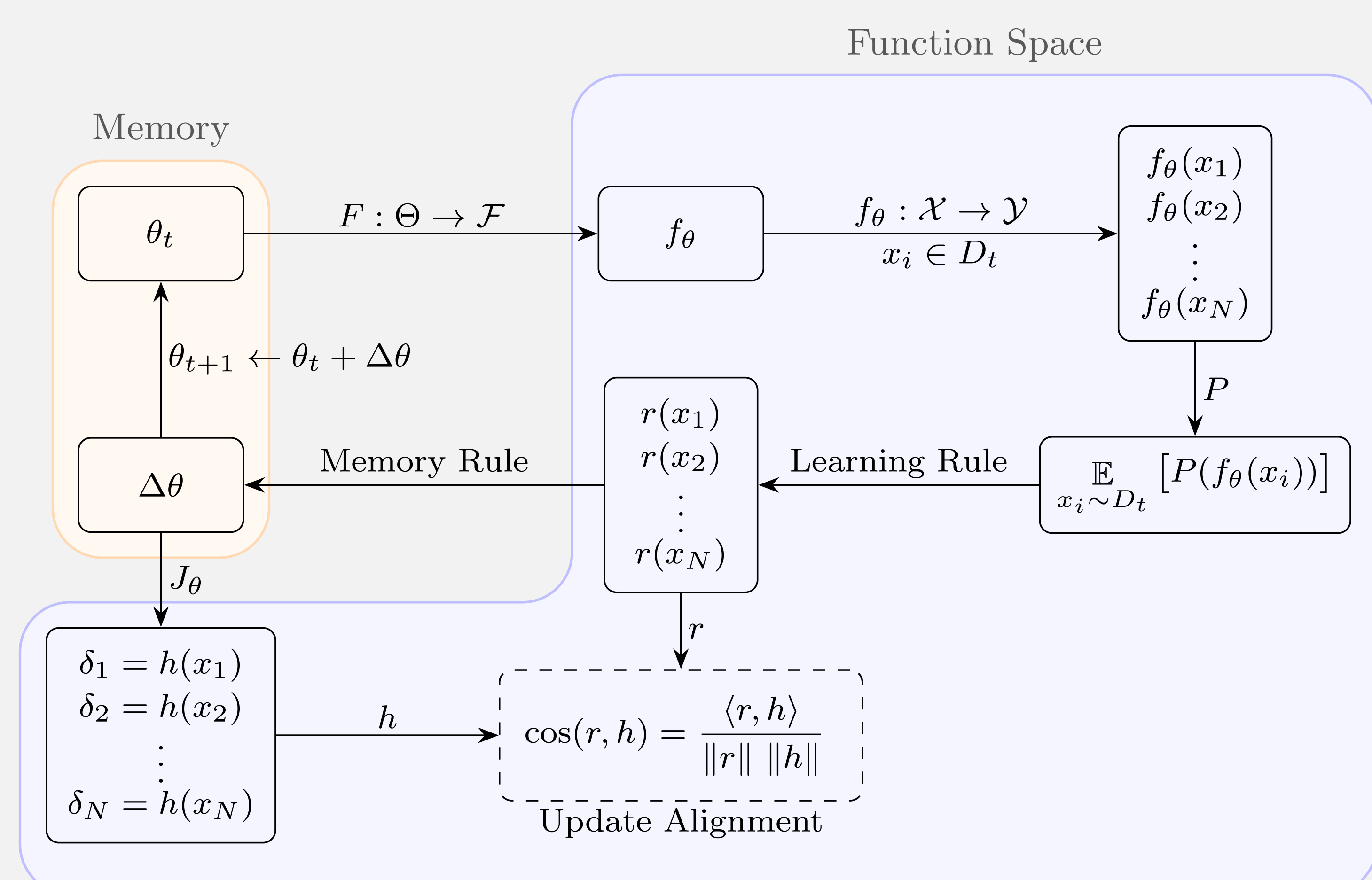
Global expressivity says what an architecture could express with *any* parameters. It says nothing about what a learner, carefully adjusting its parameters, can reach.

Learning isn't about parameters.

Feedback flows from the problem in terms of situations and actions — never parameters.

Learning signals Agents determine how to change their policies and express these as functions $c : \mathcal{X} \rightarrow \mathcal{Y}$, like the functional gradient $\nabla_f P$.

Memory is the representation of the decision rule. The memory rule receives signal c and updates parameters by $\Delta\theta$, but the resulting update $h_{\Delta\theta}$ may not be aligned with c .



Performance improvement factors

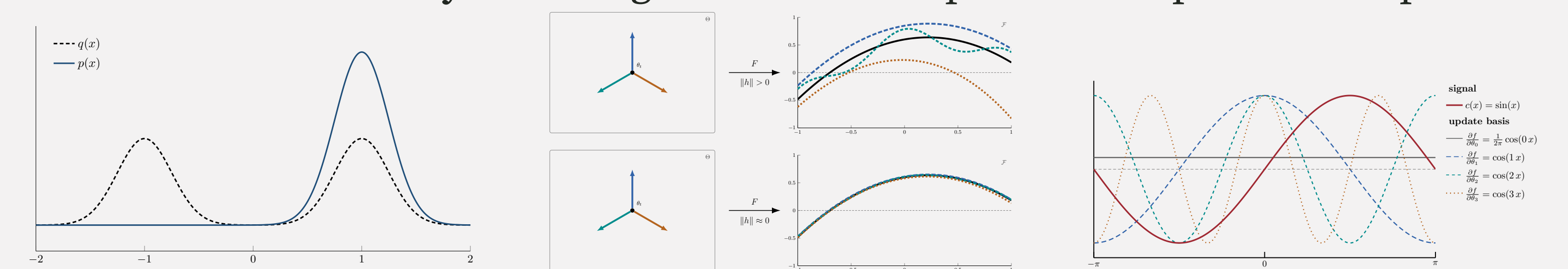
Improvement is, to first order, a product of three terms:

$$\Delta P = \underbrace{\cos(\nabla_f P, h)}_{\text{alignment}} \times \underbrace{\|h\|}_{\text{magnitude}} \times \underbrace{\|\nabla_f P(f_t)\|}_{\text{signal}} + O(\|h\|^2)$$

The agent controls magnitude and alignment, but not the gradient. When step size is limited—by engineering or physics—**alignment is the only thing left to improve.**

If you aren't learning, something is wrong

When performance is decreasing, $P(f_{t+1}) \leq P(f_t)$, there's a problem with one of the three factors — but they're entangled as a “local optimum” in parameter space:



$$\nabla_f P = 0$$

A *function space* local optimum, intrinsic to the problem.

$$\|h\| = 0$$

Parameter updates don't affect the function.

$$\cos(c, h) = 0$$

An *architecture* local optimum, intrinsic to the agent.

Learnability depends on your architecture

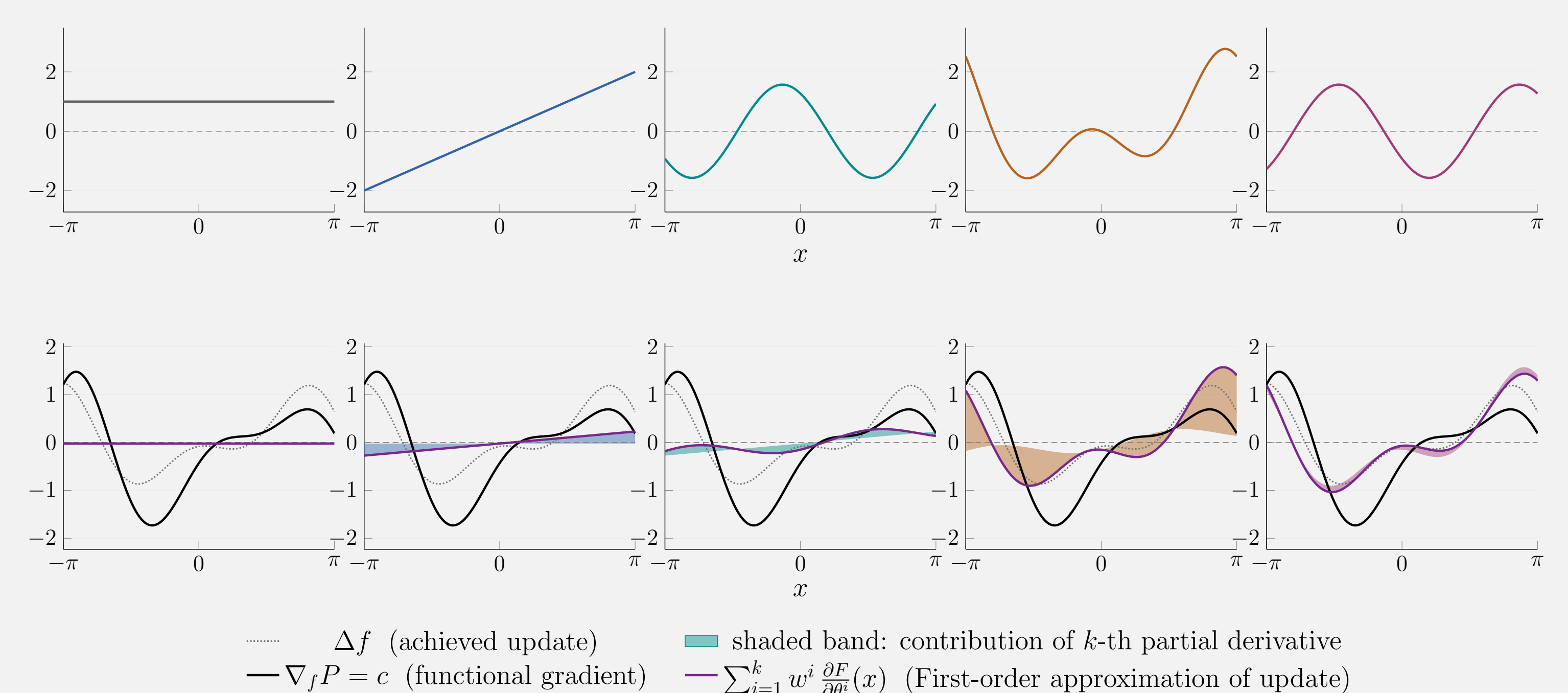
In a small step, a policy update has to lie near the architecture's tangent space:

$$f_{\theta_{t+1}} \approx f_{\theta_t} + \sum_{i=1}^{\dim(\theta)} \Delta\theta^i \frac{\partial f}{\partial \theta^i}(\theta_t)$$

The set of decision rules a small update away is an agent's **differential expressivity**.

Differential expressivity: what you can *learn*, not what exhaustive search can *find*.

Partial derivatives $\frac{\partial F}{\partial \theta^i}$ and the update direction built one parameter at a time



Differential expressivity bounds alignment: $\cos(c, h) \leq \sup_g \cos(c, g)$ over the updates g the agent can express. Natural gradients make the most of it — but differential expressivity itself can be poorly aligned, invisible to the natural metric.

Directed plasticity

Definition. The alignment $\cos(c, h)$ between a learning signal c and the update h the memory actually makes.

Improvement = signal × alignment × magnitude. Architecture constrains alignment — measure it.

